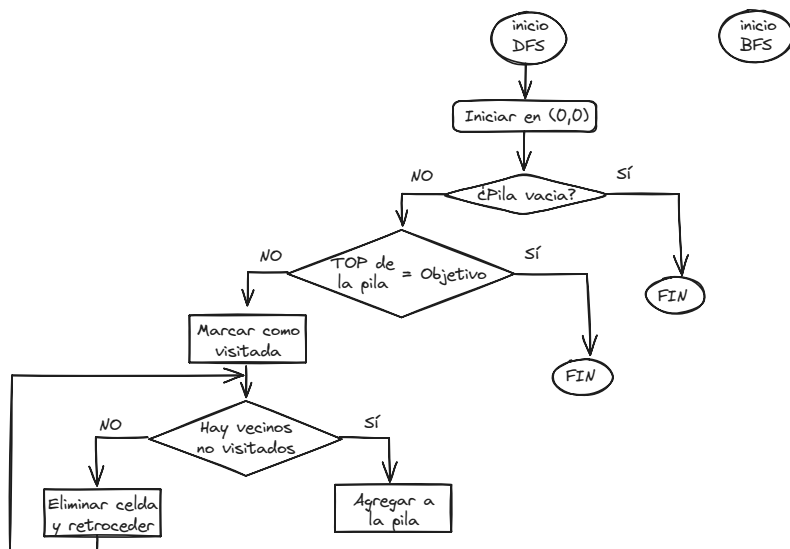
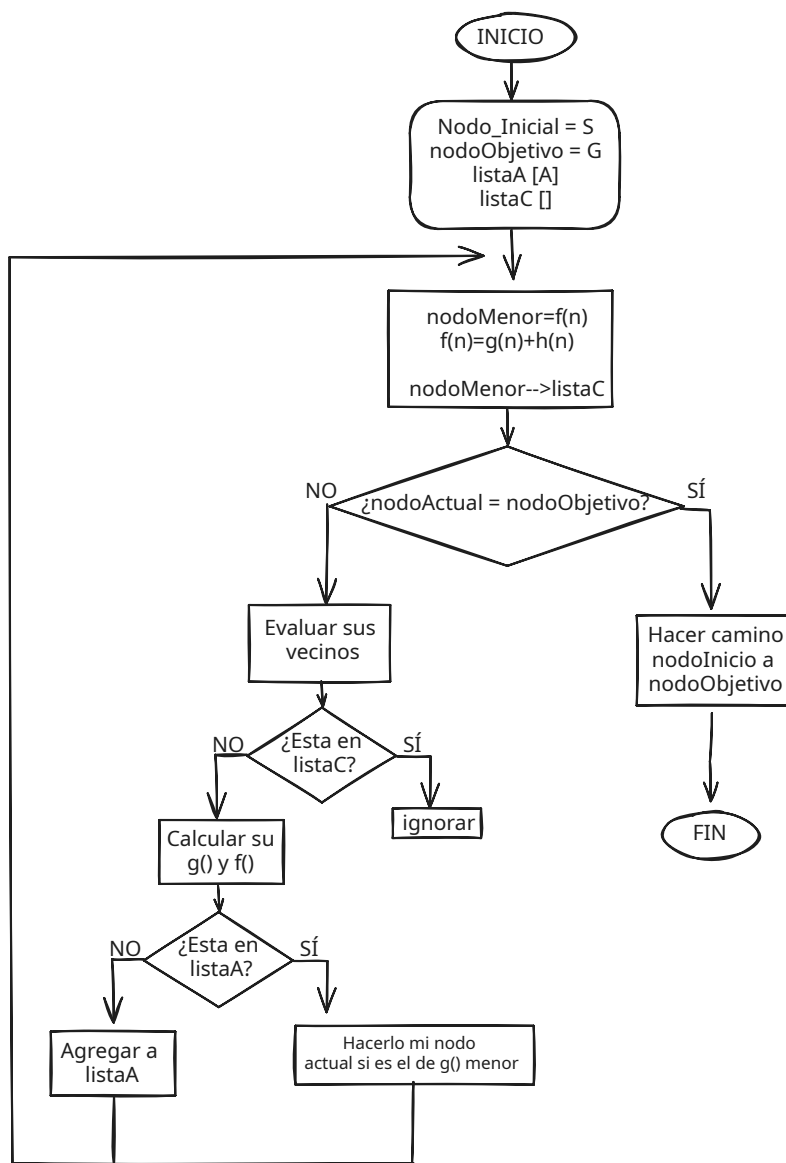


2do Parcial Practicas





1.

2.

3.

4.

Práctica 3

a) Diagrama de flujo: Algoritmo A*

-
-
- 1.
- 2. abiertos cerrados
- 3. $f(n) = g(n) + h(n)$
- 4. f(n) abiertos
- 5. abiertos

b) Gráfica del laberinto con imshow() y plot

-

```
import numpy as np
import matplotlib.pyplot as plt

laberinto = np.array([
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0],
    [1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1],
    [1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0],
    [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1],
    [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0],
])

plt.imshow(laberinto, cmap="binary")
plt.show()
```

c) Selección de coordenadas válidas

- matplotlib
-

```
def seleccionar_punto(x, y):
    if laberinto[x, y] == 0:
        plt.plot(y, x, 'o', color='red')
        plt.show()
    else:
        print("Coordenada inválida")
```

Tarea: Algoritmos BFS y DFS

1.

•
•

2.

•
•
•
•
•

Prácticas del Segundo Parcial

1.

•

2.

• H
•
•
•
• G
•

$h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 -$

$y_1)^2}$

3.

•
•

4.

•
•
•

matplotlib

Teoría

Lógica de primer orden (FOL)

-
- -
 -
 -
-
-

Padre(x, y)

Agentes inteligentes

1.

-
-

2.

-
-

3.

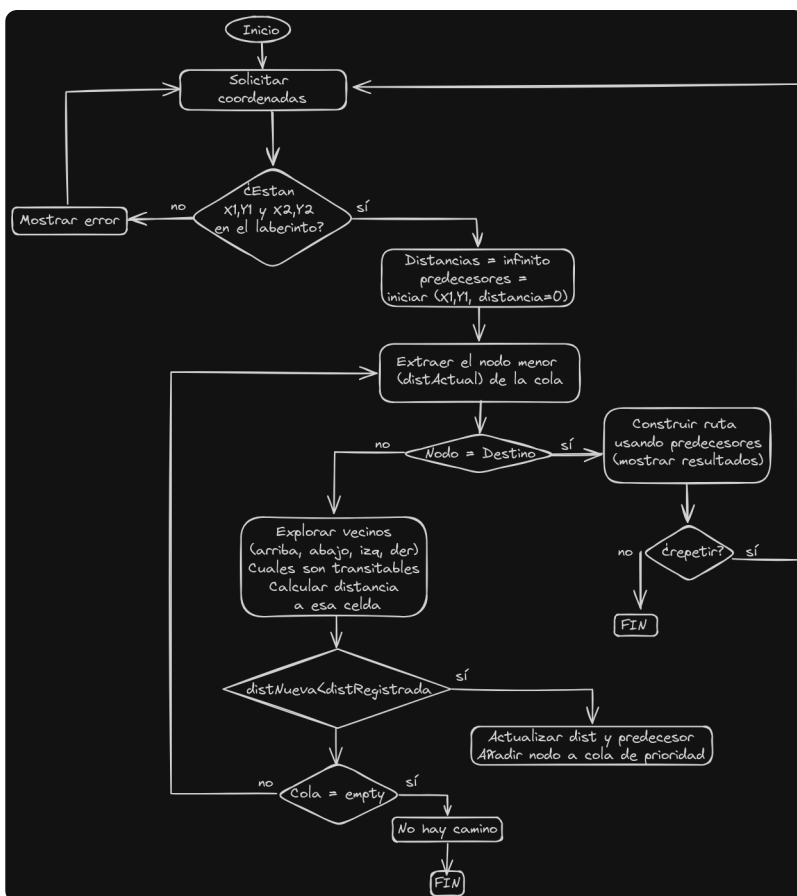
-
-

4.

-
-

Dijkstra

-
- - 1.
 - 2.
 - 3.
 - 4.



```

#Inicializar de punto inicial y final
punto_inicial = (1, 1)
meta = (14, 14)
#Tipos de movimiento
movimientos = [(-1,0), (0,1), (1,0), (0,-1)]

def dfs(laberinto, punto_inicial, meta)
    #Lista para manejar los nodos por explorar (pila)
  
```

```

pila = [(punto_inicial, [])]
#Matriz de visitados
filas = np.shape(laberinto)[0]
columnas = np.shape(laberinto)[1]
visitados = np.zeros((filas,columnas))
#Marcamos el nodo inicial como visitados
#Definir una lista que contenga a todos los nodos que he visitado
considerados = []

while len(pila) > 0:
    nodo_actual, camino = pila[-1]
    pila = pila[:1]

    #Guardar los nodos que se han ido visitando
    considerados += [considerados]

    if nodo_actual == meta
        return camino + [nodo_actual], considerados

    visitados[nodo_actual[0], nodo_actual[1]] = 1
    for direccion in movimientos:
        nueva_posicion = (nodo_actual[0] + direccion[0])
        #Ver que el vecino (nueva posición) este dentro del laberinto
        if ((0 <= nueva_posicion[0] < filas) and (0 <=
nueva_posicion[1] < columnas));
            if (maze[nueva_posicion[0], nueva_posicion[1]]) == 0 and
visitados[nueva_posicion[0], nueva_posicion[1]] == 0):

```

```

import matplotlib.pyplot as plt
import numpy as np

#Laberinto de 15x15 con 1 representado por paredes y 0 por caminos
maze = np.array([
    [1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
    [1,0,0,0,0,0,0,0,0,0,0,0,1,0,1,1,1,0,0,1],
    [1,0,0,0,1,0,1,1,1,0,1,0,1,0,0,0,0,1,1,1],
    [1,0,1,1,1,1,0,1,0,0,1,0,0,0,0,0,1,0,0,0,1],
    [1,0,0,0,0,1,0,0,1,1,0,1,1,0,0,0,0,0,1,1],
    [1,0,0,0,1,1,0,0,1,1,0,0,1,0,1,1,1,0,1,1],
    [1,0,0,1,0,1,1,1,0,1,0,0,1,0,0,0,0,0,1,1],
    [1,0,1,0,0,1,0,1,0,1,0,1,1,1,1,1,0,1,1,1],
    [1,0,0,0,0,1,1,0,1,1,0,1,1,1,1,0,0,1,0,1],
    [1,1,1,0,0,0,0,1,0,1,1,0,0,1,1,0,1,1,0,1],
    [1,0,0,1,0,0,1,0,1,0,1,1,0,0,1,0,1,1,0,1],
    [1,0,0,1,0,0,0,0,0,0,0,0,1,1,0,1,0,1,0,1],
    [1,0,0,0,1,1,0,0,0,1,0,1,0,0,1,0,0,1,0,1],
    [1,0,0,1,1,0,1,0,0,0,0,0,0,0,0,1,0,1,1,0,1],
    [1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
])

```

```

#Inicializar punto inicial y punto final
punto_inicial = (1,1)
meta = (13,15)

#Tipos de movimiento
movimientos = [(-1,0),(0,1),(1,0),(0,-1)]

def dfs(laberinto,punto_inicial,meta):
    #Lista para manejar los nodos por explorar (pila)
    pila = [(punto_inicial,[])]
    #Matriz de visitados
    filas = np.shape(laberinto)[0]
    columnas = np.shape(laberinto)[1]
    visitados = np.zeros((filas,columnas))
    #Marcamos el nodo inicial como visitado

    #Definir una lista que contenga todos los nodos que he visitado
    considerados = []

    while len(pila) > 0:
        nodo_actual,camino = pila[-1]
        pila = pila[:-1]

        #Guardar los nodos que se han ido visitando
        considerados += [nodo_actual]

        if nodo_actual == meta:
            return camino + [nodo_actual],considerados

        visitados[nodo_actual[0],nodo_actual[1]] = 1
        for direccion in movimientos:
            nueva_posicion = (nodo_actual[0] + direccion[0],nodo_actual[1]
+ direccion[1])
            #Ver que el vecino (nueva posición) esté dentro del laberinto
            if((0 <= nueva_posicion[0] < filas) and (0 <=
nueva_posicion[1] < columnas)):
                #Ver si el nodo a evaluar (nueva_posicion) es un cambio
accesible y además si ese nodo NO LO HE VISITADO
                if((maze[nueva_posicion[0],nueva_posicion[1]]) == 0 and
visitados[nueva_posicion[0],nueva_posicion[1]] == 0):
                    pila += [(nueva_posicion,camino + [nodo_actual])]
            return None,considerados

def desplegar_labderinto(maze,camino = None,considerados = None):
    plt.imshow(maze,cmap = 'binary')
    if considerados:
        for i in considerados:
            plt.plot(i[1],i[0],'o',color = 'blue')
    if camino:

```

```
        for j in camino:
            plt.plot(j[1],j[0],'o',color = 'red')
plt.show()

camino,considerados = dfs(maze,punto_inicial,meta)
desplegar_laberinto(maze,camino,considerados)
```

Lógica de Primer Orden (LPO)

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

-
-

Agentes Inteligentes

Tipos de agentes:

1.
 -
 -

1.

-

2.

-

-
-

3.

-
-

-

4.

-

-

Algoritmo de Dijkstra

1.

-

-
-

2.

-

-

3.

-

-

4.

•

•

•

•